TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Arduino Robotics & Electronics · Full-Day Camp

24 July 2026

55 children · 7 groups · full day (approx. 08:30–15:30)

Prepared for **David** FULL-DAY CAMP

Executive Summary

This is the full plan for a **Arduino Robotics & Electronics** camp of **55 children** on 6th August 2026 at Fireside Group. The intake is split into age bands — 7-9 yrs (20), 10-12 yrs (23), 13-15 yrs (12) — and each band into small groups, so every child works at the right level and gets plenty of hands-on time. The schedule, staffing, topic index and full lesson content follow below.

55	3	7	12	1:4.6	Full Day
CHILDREN	AGE BANDS	GROUPS	ADULTS	RATIO	LENGTH

Learning outcomes

- Every child builds and tests real projects pitched at their age and level.
- Children work in small teams — sharing roles, debugging together and helping one another.
- Each group presents what they made at the end-of-day showcase, building confidence and communication.

Master Day Timetable

The whole camp runs on one clock. Groups start and stop together so breaks, lunch and the showcase stay in sync.

TIME	ACTIVITY
08:30 – 09:00	Arrival, registration & name badges

TIME	ACTIVITY
09:00 – 09:20	Opening assembly — welcome, safety brief, split into groups (all bands together)
09:20 – 10:45	**Project Block 1** — each group in its own space
10:45 – 11:05	Morning break & snack (staggered so the space doesn't crowd)
11:05 – 12:30	**Project Block 2** — build continues / next activity
12:30 – 13:15	Lunch & free play (supervised, outdoors if possible)
13:15 – 14:30	**Project Block 3** — finish builds, test & debug
14:30 – 14:50	Showcase prep — groups tidy kits and rehearse a 60-second demo
14:50 – 15:20	Showcase & demos — every group presents to the whole camp
15:20 – 15:30	Closing circle, certificates & departure

Groups at a Glance

Each group stays together all day with its own lead facilitator, so no child is ever in a group larger than the target size.

AGE BAND	GROUP	SIZE
7-9 yrs	7-9 yrs · Group A	10 children
7-9 yrs	7-9 yrs · Group B	10 children
10-12 yrs	10-12 yrs · Group A	8 children
10-12 yrs	10-12 yrs · Group B	8 children
10-12 yrs	10-12 yrs · Group C	7 children
13-15 yrs	13-15 yrs · Group A	6 children
13-15 yrs	13-15 yrs · Group B	6 children

Staffing & Ratios

With **55 children** in **7 groups**, this plan needs **12 adults** on the day — an overall ratio of about **1 adult to 4.6 children** (target 1:10 or better).

ROLE	PEOPLE
Lead facilitators (one per group)	7
Floating helpers	3
Camp coordinator	1
First-aider	1
Total adults	**12**

Topics in This Plan

Each band's topics are listed by number below. A topic run by more than one band appears once in the programme (with its bands noted), so its instructions are never repeated — the same number just shows up for each band that runs it.

AGE BAND	TOPICS (BY NUMBER)
7-9 yrs 20 children	1. Traffic light simulation *(Beginner · ~60 min)* 2. Button controlled LED *(Beginner · ~45 min)* 3. Fade an LED *(Beginner · ~45 min)*
10-12 yrs 23 children	4. Capacitive touch sensor *(Beginner · ~60 min)* 5. Morse code blinker *(Intermediate · ~75 min)* 6. Temperature logger *(Intermediate · ~90 min)*
13-15 yrs 12 children	7. Fan control *(Advanced · ~90 min)* 8. Simon says game *(Advanced · ~120 min)* 9. Line-following robot *(Advanced · ~120 min)*

Lesson Topics

Each topic below is a self-contained card with its materials, step-by-step instructions and full code/build steps. Where a topic is run by more than one age band, it is shown **once** and the bands that run it are listed on the card — the instructions are the same for each, so there is no need to repeat them.

1 Traffic light simulation

BEGINNER · ~60 MIN · BANDS: 7-9 YRS (20) · 2 GROUP(S)

MATERIALS

- 1 x Arduino Uno board (Nerokas.co.ke)

- 1 x USB cable (Type A to Type B) for programming (Nerokas.co.ke)
- 1 x Breadboard, half-size or full-size (Pixel Electronics.co.ke)
- 1 x Red LED, 5mm (Nerokas.co.ke)
- 1 x Yellow (or amber) LED, 5mm (Nerokas.co.ke)
- 1 x Green LED, 5mm (Nerokas.co.ke)
- 3 x 220 ohm resistors (colour bands: red-red-brown) (Pixel Electronics.co.ke)
- 4 x Male-to-male jumper wires (Nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (not purchased locally)

HOW IT WORKS

Real traffic lights follow a fixed pattern: red means stop, green means go, and yellow (or amber) warns drivers that the light is about to change. This project recreates that pattern using three LEDs (Light Emitting Diodes) — one red, one yellow, one green — controlled by an Arduino.

An Arduino is a small computer that can turn electricity on and off very precisely, using a program (called a "sketch") that we write and upload to it. Each LED is connected to a different pin on the Arduino. In our code, we tell the Arduino things like "turn on the red LED, wait 5 seconds, turn it off, turn on the green LED..." and so on, in a loop that repeats forever.

LEDs only allow electricity to flow in one direction and they need just the right amount of current — too much and they burn out instantly. That is why each LED is paired with a 220 ohm resistor, which acts like a narrow gate that limits how much electricity can flow through, protecting the LED from damage.

The core programming idea here is **sequencing with timing**: the Arduino executes instructions one after another, and the `delay()` function tells it to pause for a set number of milliseconds before moving to the next instruction. By arranging `digitalWrite()` (turn a pin HIGH or LOW) and `delay()` commands in the right order inside a loop, we recreate the real-world timing of a traffic light.

STEP-BY-STEP INSTRUCTIONS

1. Place the breadboard in front of you with the long power rails running along the top and bottom edges.

2. Insert the red LED into the breadboard. Note that LEDs have one longer leg (the anode, positive) and one shorter leg (the cathode, negative). Place it so its two legs are in different rows of the breadboard.
3. Repeat for the yellow and green LEDs, spacing them out so each has its own set of rows and does not touch its neighbours.
4. For each LED, connect a 220 ohm resistor from the LED's shorter leg (cathode) row to the breadboard's ground (–) rail.
5. Run one jumper wire from the ground (–) rail on the breadboard to the GND pin on the Arduino.
6. Run a jumper wire from the red LED's longer leg (anode) row to digital pin 13 on the Arduino.
7. Run a jumper wire from the yellow LED's longer leg (anode) row to digital pin 12 on the Arduino.
8. Run a jumper wire from the green LED's longer leg (anode) row to digital pin 11 on the Arduino.
9. Double-check that no bare wires are touching each other on the breadboard, and that each LED has its own resistor in its path.
10. Connect the Arduino to the computer using the USB cable.
11. Open the Arduino IDE, select the correct board (Arduino Uno) and port from the Tools menu.
12. Copy the sketch from the "Full Code" section into a new sketch window.
13. Click the Upload button (right-facing arrow icon) and wait for "Done uploading" to appear.
14. Observe: the red LED should light for several seconds, then turn off as green lights up, then green turns off as yellow blinks briefly, then it returns to red — repeating in a continuous loop.
15. If an LED does not light, check that it is inserted the correct way round (long leg towards the Arduino pin side) before assuming the code is wrong.
16. Once working, ask students to time the sequence with a stopwatch and compare it to real traffic lights they have seen.

FULL CODE

```
// Traffic Light Simulation
// Three LEDs (red, yellow, green) mimic a real traffic light sequence
```

```
// Assign each LED colour to an Arduino digital pin
const int redPin = 13;     // Red LED connected to pin 13
const int yellowPin = 12;  // Yellow LED connected to pin 12
const int greenPin = 11;   // Green LED connected to pin 11

// Timing settings (in milliseconds) - easy to change later
const int redDuration = 4000;     // Red light stays on for 4 seconds
const int greenDuration = 4000;   // Green light stays on for 4 seconds
const int yellowDuration = 1000;  // Yellow light stays on for 1 second

void setup() {
  // Set all three LED pins as OUTPUT so the Arduino can send power to them
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
}

void loop() {
  // --- RED PHASE: Stop ---
  digitalWrite(redPin, HIGH);     // Turn red LED on
  digitalWrite(yellowPin, LOW);   // Make sure yellow is off
  digitalWrite(greenPin, LOW);    // Make sure green is off
  delay(redDuration);             // Wait while red stays lit

  // --- GREEN PHASE: Go ---
  digitalWrite(redPin, LOW);      // Turn red off
  digitalWrite(greenPin, HIGH);   // Turn green on
  delay(greenDuration);           // Wait while green stays lit

  // --- YELLOW PHASE: Prepare to stop ---
  digitalWrite(greenPin, LOW);    // Turn green off
  digitalWrite(yellowPin, HIGH);  // Turn yellow on as a warning
  delay(yellowDuration);          // Wait a short time on yellow
  digitalWrite(yellowPin, LOW);   // Turn yellow off before looping back to red

  // The loop() function repeats automatically, so it goes back to RED PHASE
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
An LED does not light up at all	LED inserted backwards (legs reversed)	Remove the LED, flip it around so the longer leg (anode) connects towards the Arduino pin, and reinsert
LED glows very faintly or flickers	Loose jumper wire or resistor not fully pushed into breadboard holes	Press all wires and resistor legs firmly into the breadboard rows
All LEDs stay off after uploading	Wrong board or port selected in Arduino IDE, or upload failed silently	Check Tools > Board and Tools > Port, then re-upload and wait for "Done uploading" message
Two LEDs light up at the same time when they should not		Check each phase in the code turns off the LED(s) that should no longer be lit

	digitalWrite(..., LOW) line missing or commented out for the previous colour	
Sequence timing looks wrong (too fast or too slow)	delay() values not matched to the intended real-world timing	Adjust the redDuration, greenDuration and yellowDuration constants at the top of the sketch
LED burns out (goes dark permanently, sometimes with a small pop or smell)	Missing or wrong-value resistor allowed too much current through the LED	Always use the 220 ohm resistor in series with each LED before power is applied

DISCUSSION QUESTIONS

- Why do you think traffic lights use red, yellow, and green specifically, rather than any other three colours?
- What would happen in our code if we forgot to turn off the red LED before turning on the green one?
- Why does the resistor need to be placed in the circuit, and what might happen to the LED without it?
- In the code, what is the purpose of the delay() function, and what do you think would happen if all the delay values were changed to 100 milliseconds?
- Real traffic lights sometimes have a flashing yellow warning mode at night. How could you modify this program to add that behaviour?

EXTENSION CHALLENGES

- **Easy:** Change the timing so red lasts 6 seconds and green lasts 3 seconds, then re-upload and observe the difference.
- **Medium:** Add a push button that, when pressed, immediately switches the light to red (like a pedestrian crossing button), then resumes the normal sequence afterward.
- **Hard:** Build a second traffic light for a perpendicular road using three more LEDs, and write code so that when one light is green, the other is red, simulating a real road intersection.

2 Button controlled LED

BEGINNER · ~45 MIN · BANDS: 7-9 YRS (20) · 2 GROUP(S)

MATERIALS

- 1 x Arduino Uno board (Nerokas, nerokas.co.ke)

- 1 x USB cable (Type A to Type B) for Arduino Uno (Nerokas, nerokas.co.ke)
- 1 x Breadboard (half-size or full-size) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x LED (any colour, 5mm) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x Push button (4-leg tactile switch) (Nerokas, nerokas.co.ke)
- 1 x 220 ohm resistor (for the LED) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x 10k ohm resistor (for the button, optional if using internal pull-up in code) (Pixel Electronics, pixelelectronics.co.ke)
- 6-8 x Male-to-male jumper wires (Nerokas, nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (school-provided or student's own)

HOW IT WORKS

An LED is a small light that only lets electricity flow through it in one direction. When electricity flows, it lights up. A push button is just a switch — when you press it, it closes a gap so electricity can flow through it; when you let go, the gap opens again and electricity stops flowing through that path.

Normally, if we wired a button straight to an LED, the LED would only be on while your finger is pressing the button, and off the moment you let go. That is called "momentary" control. But in this project, we want something smarter: press the button once and the LED turns ON and stays on, even after you take your finger off. Press it again and the LED turns OFF and stays off. This is called "toggling" — like a light switch on a wall that stays in whichever position you last flipped it to.

The Arduino makes this possible because it has a tiny brain (a microcontroller) that can remember things. Every time the code notices that the button has just been pressed, it flips a stored value in its memory from "LED is off" to "LED is on" or back again. The Arduino checks the button thousands of times per second, so it reacts almost instantly, but it only flips the LED state once per press — not for the whole time your finger is down. That is the key programming idea in this project: detecting the moment of a "new press" rather than just checking if the button happens to be down right now.

STEP-BY-STEP INSTRUCTIONS

1. Place the breadboard in front of you and plug the Arduino Uno into the USB cable, connecting it to the computer.

2. Insert the LED into the breadboard so its two legs are in different rows. Note that the longer leg is the positive leg (anode) and the shorter leg is the negative leg (cathode).
3. Connect the 220 ohm resistor from the LED's longer leg (anode) to an empty row on the breadboard — this resistor protects the LED from too much current.
4. Run a jumper wire from the other end of the 220 ohm resistor to digital pin 8 on the Arduino.
5. Run a jumper wire from the LED's shorter leg (cathode) to the GND (ground) rail on the breadboard, and connect that rail to a GND pin on the Arduino.
6. Insert the push button into the breadboard so it straddles the centre gap (this ensures its two sides are not connected to each other by default).
7. Connect one leg of the button to digital pin 2 on the Arduino.
8. Connect the same leg's neighbouring pin (the other pin on that side of the button) to GND, using the 10k ohm resistor if you are not using the Arduino's internal pull-up resistor in code (see code comments for the alternative wiring using INPUT_PULLUP, which needs no extra resistor).
9. Double-check all connections against the circuit diagram before powering on — loose wires are the most common cause of problems.
10. Open the Arduino IDE on the computer, create a new sketch, and type or paste the code from the "Full Code" section below.
11. Select the correct board (Arduino Uno) and port from the Tools menu in the IDE.
12. Click Upload and wait for the "Done uploading" message.
13. Press the push button once and observe: the LED should turn ON and remain on even after releasing the button.
14. Press the push button again and observe: the LED should turn OFF and remain off.
15. Ask students to press the button rapidly several times and discuss why the LED still toggles correctly once per press rather than flickering.

FULL CODE

```
// Button controlled LED - toggles LED on/off with each press

const int buttonPin = 2;   // Pin connected to the push button
const int ledPin = 8;      // Pin connected to the LED (through resistor)

int ledState = LOW;            // Current state of the LED, starts OFF
```

```
int lastButtonState = HIGH;    // Previous reading from the button pin
int currentButtonState;        // Current reading from the button pin

// Variables for simple debounce (ignoring electrical noise from the switch)
unsigned long lastDebounceTime = 0;   // Last time the button state changed
unsigned long debounceDelay = 50;     // Wait time in milliseconds to confirm a stable press

void setup() {
  pinMode(buttonPin, INPUT_PULLUP);  // Use Arduino's internal pull-up resistor
                                     // This means the button reads HIGH when NOT pressed,
                                     // and LOW when pressed (no external 10k resistor needed
                                     // if wired this way - connect the other button leg to GND)
  pinMode(ledPin, OUTPUT);           // Set LED pin as an output
  digitalWrite(ledPin, ledState);    // Make sure LED starts OFF
}

void loop() {
  int reading = digitalRead(buttonPin); // Read the current button state

  // If the reading changed, it might be noise or a real press - reset the timer
  if (reading != lastButtonState) {
    lastDebounceTime = millis(); // Record the time of this change
  }

  // If enough time has passed since the last change, we trust the reading is stable
  if ((millis() - lastDebounceTime) > debounceDelay) {

    // Only act if the stable reading is different from what we currently think
    if (reading != currentButtonState) {
      currentButtonState = reading; // Update our trusted button state

      // A press is detected when the button reads LOW (because of INPUT_PULLUP)
      if (currentButtonState == LOW) {
        ledState = !ledState;           // Flip the LED state (on becomes off, off becomes on)
        digitalWrite(ledPin, ledState); // Send the new state to the LED
      }
    }
  }

  lastButtonState = reading; // Store the reading for comparison next time through the loop
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
LED never lights up at all	LED inserted backwards (anode/cathode reversed), or wire in wrong breadboard row	Flip the LED around so the longer leg connects toward pin 8 through the resistor; check breadboard rows are correctly aligned
LED flickers or toggles multiple times per single press	Switch "bounce" - the mechanical button contacts vibrate briefly when pressed, sending several rapid signals	Make sure the debounce code (debounceDelay) is included and unchanged; increase debounceDelay to 75-100ms if it still flickers
	Button wiring pins are on the same side of the breadboard gap, so it is	

Button seems to do nothing / LED state never changes	always connected (or always disconnected)	Reposition the button so it straddles the centre gap of the breadboard, using pins from both sides
LED is very dim or barely visible	Resistor value too high, or LED wired without enough current	Confirm a 220 ohm resistor is used (not a much larger value like 10k), and check all connections are firm
Code uploads but nothing happens at all	Wrong board or port selected in Arduino IDE, or wrong pin numbers used in code vs wiring	Check Tools > Board and Tools > Port settings match the connected Arduino; confirm buttonPin and ledPin numbers match actual wiring

DISCUSSION QUESTIONS

- What is the difference between a button that only lights the LED while held down, and one that toggles the LED on and off?
- Why does the Arduino need to "remember" whether the LED is currently on or off?
- What do you think would happen if we removed the debounce code from the program? Why might that cause problems?
- Why is a resistor needed for the LED but the button uses the Arduino's built-in pull-up resistor instead of needing its own separate one?
- Can you think of a real-world device that uses a "toggle" button like this one (press once for on, press again for off)?

EXTENSION CHALLENGES

- **Easy:** Add a second LED that turns on whenever the first LED is off, and vice versa (so exactly one LED is lit at all times).
- **Medium:** Modify the code so that instead of just toggling on/off, each button press cycles through three states: LED off, LED dim (using PWM/analogWrite), and LED full brightness.
- **Hard:** Add a second button that resets the LED to OFF regardless of its current state, and use a piezo buzzer to make a short beep every time the LED state changes.

3 Fade an LED

BEGINNER · ~45 MIN · BANDS: 7-9 YRS (20) · 2 GROUP(S)

MATERIALS

- 1 x Arduino Uno (or compatible board) (Nerokas, nerokas.co.ke)

- 1 x USB cable (A to B) for programming the Arduino (Nerokas, nerokas.co.ke)
- 1 x Breadboard (half-size or full-size) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x LED (any colour, standard 5mm) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x 220 ohm resistor (Nerokas, nerokas.co.ke)
- 2 x Male-to-male jumper wires (Nerokas, nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed

HOW IT WORKS

An LED is either fully on or fully off — it doesn't understand "half power" the way a light bulb with a dimmer switch does. But we can trick our eyes into seeing a smooth fade by turning the LED on and off very, very quickly. This is called PWM, which stands for Pulse Width Modulation. Think of it like flicking a light switch on and off so fast that instead of seeing flickering, your eyes blend it into one smooth brightness level.

If the LED is on for a long time and off for a short time during each tiny cycle, it looks bright. If it's on for a short time and off for a long time, it looks dim. The Arduino does this flicking automatically for us using a special command called `analogWrite()`. We give it a number between 0 (always off) and 255 (always on), and it handles the fast switching in the background — around 490 times every second, way faster than our eyes can notice.

Not every pin on the Arduino can do this trick. Only pins marked with a squiggly line (~) next to their number — like pin 9 — support PWM. That's why we must plug our LED into one of those special pins.

STEP-BY-STEP INSTRUCTIONS

1. Place the LED on the breadboard so its two legs are in different rows. Note that one leg is longer (the positive leg, called the anode) and one is shorter (the negative leg, called the cathode).
2. Insert the 220 ohm resistor so it connects the LED's short leg (cathode) row to one of the breadboard's ground rails.
3. Use a jumper wire to connect that ground rail to the GND pin on the Arduino.
4. Use a second jumper wire to connect the LED's long leg (anode) row directly to pin 9 on the Arduino (look for the ~ symbol next to pin 9 — this confirms it supports PWM).

5. Double-check the wiring: LED long leg to pin 9, LED short leg through the resistor to GND. Wiring the LED backwards will simply stop it lighting up — it will not damage anything.
6. Connect the Arduino to the computer using the USB cable.
7. Open the Arduino IDE, select the correct board (Tools > Board) and port (Tools > Port).
8. Type or paste the sketch from the Full Code section below into a new sketch window.
9. Click Upload and wait for the "Done uploading" message.
10. Observe the LED: it should smoothly brighten from off to fully bright, then smoothly dim back down to off, repeating continuously.
11. Ask students to try changing the delay value in the code to make the fade faster or slower, then re-upload and observe the difference.
12. Challenge students to predict what will happen if they change 255 to a smaller number like 150, then test their prediction.

FULL CODE

```
// Fade an LED smoothly using PWM (Pulse Width Modulation)

int ledPin = 9;        // LED is connected to pin 9, which supports PWM (marked with ~)
int brightness = 0;    // Starting brightness level (0 = off)
int fadeAmount = 5;    // How much to change the brightness each step

void setup() {
  pinMode(ledPin, OUTPUT); // Set pin 9 as an output so it can send power to the LED
}

void loop() {
  analogWrite(ledPin, brightness); // Set the LED brightness (0 = off, 255 = fully on)

  brightness = brightness + fadeAmount; // Increase brightness by fadeAmount each loop

  // Reverse the direction of fading once we hit the top or bottom
  if (brightness <= 0 || brightness >= 255) {
    fadeAmount = -fadeAmount; // Flip the sign so brightness starts decreasing instead of increasing (or
vice versa)
  }

  delay(30); // Wait 30 milliseconds before the next brightness step, so the fade is smooth and visible
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX

LED does not light up at all	LED inserted backwards, or connected to a non-PWM pin	Flip the LED around so the long leg faces pin 9, and confirm pin 9 has a ~ symbol next to it
LED flickers instead of fading smoothly	Delay value is too large, making brightness steps too visible	Reduce the delay to something like 15-30 milliseconds
Upload fails with a "port not found" or similar error	Wrong board or port selected in the Arduino IDE, or USB cable is faulty/ power-only	Check Tools > Board and Tools > Port, and try a different USB cable known to transfer data
LED stays at full brightness or fully off, no fading	Using digitalWrite() instead of analogWrite(), or LED plugged into a non-PWM pin	Check the code uses analogWrite(), and confirm the pin number matches a PWM-capable pin
LED is very dim even at brightness 255	Resistor value is too high, or LED is a low-brightness type	Try a lower value resistor (e.g. 220 ohm instead of 1k ohm) if available, or test with a different LED

DISCUSSION QUESTIONS

- What does PWM stand for, and what job is it actually doing behind the scenes?
- Why can't we plug the LED into just any pin on the Arduino if we want to fade it?
- What do you think would happen to the fade if we used a delay of 200 milliseconds instead of 30? Why?
- Can you think of another everyday device that might use a similar "trick the eye" technique to control brightness or speed?
- What is the purpose of the fadeAmount variable becoming negative partway through the program?

EXTENSION CHALLENGES

- **Easy:** Change the code so the LED fades faster on the way up and slower on the way down.
- **Medium:** Add a second LED on a different PWM pin and make it fade in the opposite direction (bright when the first is dim, and vice versa).
- **Hard:** Use a potentiometer (variable resistor) connected to an analog input pin to let a student manually control the brightness of the LED in real time, instead of the automatic fade.

4 Capacitive touch sensor

BEGINNER · ~60 MIN · BANDS: 10-12 YRS (23) · 3 GROUP(S)

MATERIALS

- 1 x Arduino Uno board (Nerokas, nerokas.co.ke)
- 1 x USB cable (Type A to Type B) for programming the Arduino (Nerokas, nerokas.co.ke)
- 1 x TTP223 capacitive touch sensor module (Pixel Electronics, pixelelectronics.co.ke)
- 1 x LED (5mm, any colour, e.g. red) (Nerokas, nerokas.co.ke)
- 1 x 220 ohm resistor (current-limiting resistor for the LED) (Nerokas, nerokas.co.ke)
- 1 x Half-size or full-size breadboard (Pixel Electronics, pixelelectronics.co.ke)
- 6 x Male-to-male jumper wires (Nerokas, nerokas.co.ke)
- 1 x USB power bank or laptop (for powering the Arduino during the lesson) (not supplied by vendor — use existing classroom equipment)

HOW IT WORKS

A capacitive touch sensor can tell when a human finger is nearby, without any physical button being pressed down. Our bodies naturally hold a tiny electrical charge. The touch sensor module has a small metal pad that is sensitive to this charge. When your finger gets close to or touches the pad, it changes the electrical field around the pad very slightly. The chip on the module (called a TTP223) notices this change and immediately sends out a clear, easy-to-read signal: either HIGH (which means "yes, someone is touching me") or LOW (which means "no touch detected").

Think of it like a doorbell that rings not because you pressed a physical button, but because you got close enough for it to notice you were there. The Arduino constantly "listens" to this signal on one of its digital pins. In our project, every time the sensor changes from "not touched" to "touched," the Arduino code flips the state of an LED — if the LED was off, it turns on; if it was on, it turns off. This is called "toggling," and it is different from just holding a button down to keep a light on. To make toggling work reliably, the code needs to notice the exact moment the touch begins, not repeat the toggle over and over while your finger stays on the

pad. This is done by comparing the sensor's current reading to what it was a moment ago — a technique called edge detection.

STEP-BY-STEP INSTRUCTIONS

1. Introduce the TTP223 touch sensor module to students. Point out its three pins: VCC (power), GND (ground), and OUT (output/signal). Explain that it also has a small touch pad marked on the board.
2. Place the LED on the breadboard, making sure the two legs are in different rows. Identify the longer leg (positive/anode) and the shorter leg (negative/cathode).
3. Connect the 220 ohm resistor in series with the LED: one end of the resistor to the LED's longer leg's row, and the other end of the resistor to an empty row that will connect to an Arduino digital pin.
4. Connect the LED's shorter leg (cathode) to the breadboard's ground rail.
5. Run a jumper wire from the breadboard's ground rail to the Arduino's GND pin.
6. Run a jumper wire from the row connected to the resistor to Arduino digital pin 8 (this will control the LED).
7. Connect the touch sensor module's VCC pin to the Arduino's 5V pin using a jumper wire.
8. Connect the touch sensor module's GND pin to the Arduino's GND pin (or to the breadboard ground rail, which is already connected to GND).
9. Connect the touch sensor module's OUT pin to Arduino digital pin 2 using a jumper wire.
10. Double-check all connections before powering on: VCC to 5V, GND to GND, OUT to pin 2, LED circuit to pin 8 and GND through the resistor.
11. Connect the Arduino to the computer using the USB cable. Open the Arduino IDE and select the correct board and port from the Tools menu.
12. Copy the full code (below) into a new sketch, then click Upload. Wait for the "Done uploading" message.
13. Ask students to touch the sensor pad once and observe: the LED should turn on. Ask them to touch it again and observe the LED turning off.
14. Discuss what they notice: does the LED stay on only while the finger is touching, or does it stay on after they let go? (It should stay on/off — that's the toggle behaviour.)
15. Encourage students to experiment with touching quickly versus slowly and note whether the toggle still works reliably each time.

FULL CODE

```
// Capacitive Touch Sensor - LED Toggle Project
// The LED changes state (on/off) each time the touch sensor is touched

const int touchPin = 2;   // Digital pin connected to the touch sensor's OUT pin
const int ledPin = 8;     // Digital pin connected to the LED (through the resistor)

bool ledState = false;        // Tracks whether the LED is currently on or off
int touchState = 0;            // Stores the current reading from the touch sensor
int lastTouchState = 0;        // Stores the previous reading, used to detect a new touch

void setup() {
  pinMode(touchPin, INPUT);   // Set the touch sensor pin as an input
  pinMode(ledPin, OUTPUT);    // Set the LED pin as an output
  digitalWrite(ledPin, LOW);  // Make sure the LED starts off
  Serial.begin(9600);         // Start serial communication for debugging
}

void loop() {
  touchState = digitalRead(touchPin); // Read the current state of the touch sensor (HIGH or LOW)

  // Check if the sensor has just changed from not-touched (LOW) to touched (HIGH)
  // This "edge detection" stops the LED from flickering rapidly while the finger stays on the pad
  if (touchState == HIGH && lastTouchState == LOW) {
    ledState = !ledState;              // Flip the LED state: true becomes false, false becomes true
    digitalWrite(ledPin, ledState);    // Apply the new state to the LED pin
    Serial.println(ledState ? "LED ON" : "LED OFF"); // Print the new state to the Serial Monitor for
debugging

    delay(50); // Short delay to help avoid false re-triggers right after a touch (basic debounce)
  }

  lastTouchState = touchState; // Save the current reading so we can compare it next time through the loop
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
LED never turns on, no matter how the sensor is touched	OUT pin of the sensor connected to the wrong Arduino pin, or code using a different pin number than wired	Check that the OUT pin is on digital pin 2 and that `touchPin` in the code matches the wiring
LED flickers rapidly or toggles multiple times with a single touch	No debounce, or edge detection not implemented, so the fast loop catches "touched" state many times	Confirm the code compares `touchState` to `lastTouchState` and includes the short delay after a toggle
LED is always on, even without touching the sensor	LED wired directly to 5V instead of a digital pin, or resistor missing causing a short/bright always-on LED	Check the LED's longer leg connects through the 220 ohm resistor to pin 8, not directly to 5V

Sensor seems to trigger on its own, or the LED toggles without anyone touching it	Loose jumper wires causing noise, or hand/nearby metal objects close to the touch pad	Reseat all jumper wires firmly, keep hands and metal objects away from the pad when not testing
Nothing happens and Serial Monitor shows nothing	Wrong baud rate selected in Serial Monitor, or wrong board/port selected in Arduino IDE	Set Serial Monitor baud rate to 9600 and confirm correct board/port under Tools menu

DISCUSSION QUESTIONS

- What is different between a normal push-button switch and a capacitive touch sensor?
- Why does the code need to remember the previous touch state (`lastTouchState`) instead of just checking the current state?
- What might happen if we removed the delay after detecting a touch? Why might that cause problems?
- Can you think of a real-world device you have seen or used that uses capacitive touch sensing?
- Why do you think our bodies are able to trigger the sensor just by getting close, without needing to press hard like a button?

EXTENSION CHALLENGES

- **Easy:** Change the code so touching the sensor makes the LED blink three times quickly instead of just turning on or off.
- **Medium:** Add a second LED and modify the code so each touch cycles through three states: both LEDs off, only LED 1 on, both LEDs on.
- **Hard:** Use the touch sensor to control the brightness of an LED with PWM (analogWrite), so each touch increases brightness in steps, cycling back to off after the maximum brightness is reached.

5 Morse code blinker

INTERMEDIATE · ~75 MIN · BANDS: 10-12 YRS (23) · 3 GROUP(S)

MATERIALS

- 1 x Arduino Uno (or compatible clone) (Nerokas, nerokas.co.ke)
- 1 x USB cable (A to B) for programming the Arduino (Nerokas, nerokas.co.ke)
- 1 x Breadboard (half-size or full-size) (Pixel Electronics, pixelelectronics.co.ke)

- 1 x LED (5mm, any colour, red or green recommended for visibility) (Nerokas, nerokas.co.ke)
- 1 x 220 ohm resistor (for current limiting) (Nerokas, nerokas.co.ke)
- 2 x Jumper wires (male-to-male) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x Push button (optional, for triggering messages) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x 10k ohm resistor (only needed if using the optional push button) (Nerokas, nerokas.co.ke)
- 1 x Laptop or computer with the Arduino IDE installed (shared classroom resource)

HOW IT WORKS

Long before phones and the internet, people sent messages over long distances using electrical signals. They used a system called Morse code, invented by Samuel Morse. Instead of typing letters, you send short flashes of light or sound (called "dots") and long flashes (called "dashes"). Every letter of the alphabet has its own unique pattern of dots and dashes. For example, the letter S is three dots (dot-dot-dot) and the letter O is three dashes (dash-dash-dash) — that's where the famous "SOS" distress signal comes from.

In this project, we use an LED (a small light) connected to the Arduino to "speak" Morse code. The Arduino is a tiny computer that can turn the LED on and off very precisely, for exact amounts of time. We write a program that tells the Arduino: "turn the LED on for a short time for a dot, or a longer time for a dash, then turn it off for a little gap before the next signal." By stringing together dots and dashes with the right timing, the LED can blink out entire words and sentences.

The key electronics idea here is timing control using the `delay()` function, and the key programming idea is breaking a big problem (spelling a whole message) into small, repeatable pieces (looking up each letter's pattern and blinking it out). This is exactly how real telegraph operators and radio operators used timing and rhythm to communicate before modern technology existed.

STEP-BY-STEP INSTRUCTIONS

1. Gather all materials and place the breadboard in front of you with the Arduino next to it.

2. Insert the LED into the breadboard. Note which leg is longer (the positive leg, called the anode) and which is shorter (the negative leg, called the cathode).
3. Insert the 220 ohm resistor so that one end connects to the LED's longer leg (anode) and the other end sits in an empty row of the breadboard.
4. Use a jumper wire to connect the free end of the resistor to digital pin 9 on the Arduino.
5. Use a second jumper wire to connect the LED's shorter leg (cathode) to the GND (ground) pin on the Arduino.
6. Double-check the wiring: LED anode → resistor → Arduino pin 9, and LED cathode → Arduino GND. Ask a teacher or peer to verify before powering on.
7. Connect the Arduino to the computer using the USB cable. Open the Arduino IDE.
8. Copy the full code from the "Full Code" section into a new sketch in the Arduino IDE.
9. Select the correct board (Tools > Board > Arduino Uno) and the correct port (Tools > Port).
10. Click the Upload button. Students should observe the small "L" LED on the Arduino board blinking briefly — this means code is transferring.
11. Once uploaded, watch the external LED on the breadboard. It should begin blinking out the message "SOS" in Morse code: three short blinks, three long blinks, three short blinks, then a pause before repeating.
12. Ask students to time the blinks with a phone stopwatch or by counting "one-Mississippi" to notice the difference between a dot (short) and a dash (three times as long).
13. Challenge students to look at the code's `message` variable and change it to their own name or a short word, then re-upload and observe the new pattern.
14. Discuss what happens if a letter in their message isn't in the Morse code lookup table (it should be skipped or produce a gap — this is a good moment to explore the code's limitations together).

FULL CODE

```
// Morse Code Blinker
// Blinks out a message in Morse code using an LED connected to pin 9

const int ledPin = 9;      // The pin connected to our LED through the resistor
```

```
// Timing unit in milliseconds - all Morse timing is based on multiples of this
const int unitTime = 200;  // One "unit" = 200ms. Change this to speed up or slow down the whole message

// The message we want to blink out - only letters and numbers, no punctuation
String message = "SOS";

// This function runs once when the Arduino powers on or resets
void setup() {
  pinMode(ledPin, OUTPUT);   // Tell the Arduino that pin 9 will send power out, not read input
  Serial.begin(9600);        // Start serial communication so we can see debug messages on the computer
}

// This function repeats forever after setup() finishes
void loop() {
  blinkMessage(message);  // Blink out the whole message once
  delay(unitTime * 7);    // Wait a long pause (7 units) before repeating the message
}

// Takes a whole message (word or sentence) and blinks out each letter in turn
void blinkMessage(String msg) {
  msg.toUpperCase(); // Convert everything to uppercase since our lookup table only has capital letters

  for (int i = 0; i < msg.length(); i++) {  // Go through each character in the message one at a time
    char c = msg.charAt(i);                // Get the current character

    if (c == ' ') {
      delay(unitTime * 7);  // A space between words gets a longer pause (7 units)
    } else {
      String pattern = getMorsePattern(c); // Look up the dot/dash pattern for this letter
      blinkPattern(pattern);                // Blink out that pattern on the LED
      delay(unitTime * 3);                  // Pause 3 units between letters (standard Morse spacing)
    }
  }
}

// Blinks a single pattern of dots and dashes, e.g. "..." for S
void blinkPattern(String pattern) {
  for (int i = 0; i < pattern.length(); i++) {  // Go through each symbol in the pattern
    char symbol = pattern.charAt(i);

    if (symbol == '.') {
      digitalWrite(ledPin, HIGH); // Turn LED on
      delay(unitTime);            // Keep it on for 1 unit - this is a "dot"
      digitalWrite(ledPin, LOW);  // Turn LED off
    } else if (symbol == '-') {
      digitalWrite(ledPin, HIGH); // Turn LED on
      delay(unitTime * 3);        // Keep it on for 3 units - this is a "dash"
      digitalWrite(ledPin, LOW);  // Turn LED off
    }

    delay(unitTime); // Short pause (1 unit) between symbols within the same letter
  }
}

// Returns the Morse code pattern (as a String of dots and dashes) for a given letter or number
// If the character isn't found, returns an empty string (the LED will just skip it)
String getMorsePattern(char c) {
  switch (c) {
    case 'A': return ".-";
    case 'B': return "-...";
```

```
    case 'C': return "-.-.";
    case 'D': return "-..";
    case 'E': return ".";
    case 'F': return "..-.";
    case 'G': return "--.";
    case 'H': return "....";
    case 'I': return "..";
    case 'J': return ".---";
    case 'K': return "-.-";
    case 'L': return ".-..";
    case 'M': return "--";
    case 'N': return "-.";
    case 'O': return "---";
    case 'P': return ".--.";
    case 'Q': return "--.-";
    case 'R': return ".-.";
    case 'S': return "...";
    case 'T': return "-";
    case 'U': return "..-";
    case 'V': return "...-";
    case 'W': return ".--";
    case 'X': return "-..-";
    case 'Y': return "-.--";
    case 'Z': return "--..";
    case '0': return "-----";
    case '1': return ".----";
    case '2': return "..---";
    case '3': return "...--";
    case '4': return "....-";
    case '5': return ".....";
    case '6': return "-....";
    case '7': return "--...";
    case '8': return "---..";
    case '9': return "----.";
    default: return "";  // Unknown character - skip it (produces no blink, just the normal letter gap)
  }
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
LED does not light up at all	LED inserted backwards, or wired to the wrong pins	Check that the longer leg (anode) connects through the resistor to pin 9, and the shorter leg (cathode) connects to GND. Try flipping the LED around.
LED stays on constantly, never blinks	Code uploaded successfully but wiring bypasses pin control, or an old sketch with different logic is still running	Re-check pin 9 connection, confirm the correct sketch was uploaded, and press the Arduino's reset button.
Upload fails with a "port not found" or similar error	Wrong board or port selected in the Arduino IDE, or USB cable is loose/ faulty	Go to Tools > Board and Tools > Port and select the correct Arduino and COM port; try a different USB cable or port.

Message blinks too fast or too slow to follow	The unitTime value is not suited to the audience	Increase unitTime (e.g. to 300 or 400) for a slower, easier-to-follow pattern, or decrease it for a faster pattern.
Changing the message variable doesn't change the blinking	Students edited the code but forgot to re-upload it to the Arduino	Remind students to click Upload again after every code change — the Arduino only runs the last program that was uploaded.
LED is very dim	Resistor value is too high, or a loose connection on the breadboard	Confirm a 220 ohm resistor is used and check that all jumper wires and legs are firmly pushed into the breadboard holes.

DISCUSSION QUESTIONS

- What is the difference between a "dot" and a "dash" in Morse code, and how does our code represent that difference using time?
- Why do you think Morse code puts longer pauses between words than between letters, and between letters than between individual dots and dashes?
- Before radios and telephones, why might a simple on/off light signal have been such a powerful way to communicate?
- What would happen to the blinking pattern if we changed the unitTime variable from 200 to 100? Why?
- Can you think of another way (besides light) that this same code could be adapted to send Morse code — for example using sound or vibration?

EXTENSION CHALLENGES

- **Easy:** Change the message variable to spell your own name or your school's name, then re-upload and time how long the full message takes to blink once.
- **Medium:** Add a push button so that the message only starts blinking when the button is pressed, instead of repeating automatically forever.
- **Hard:** Modify the code so that a buzzer beeps in time with the LED (dots and dashes as short and long beeps), so the message can be both seen and heard at the same time.

6 Temperature logger

INTERMEDIATE · ~90 MIN · BANDS: 10-12 YRS (23) · 3 GROUP(S)

MATERIALS

- 1 x Arduino Uno board (Nerokas, nerokas.co.ke)
- 1 x USB cable (A to B) for Arduino Uno (Nerokas, nerokas.co.ke)

- 1 x DHT11 Temperature & Humidity Sensor module (with breakout board and 3 pins) (Pixel Electronics, pixelelectronics.co.ke)
- 1 x Half-size breadboard (Nerokas, nerokas.co.ke)
- 1 x 10kΩ resistor (only needed if using a bare 4-pin DHT11 without a breakout board) (Pixel Electronics, pixelelectronics.co.ke)
- 3 x Male-to-male jumper wires (Nerokas, nerokas.co.ke)
- 1 x 16x2 LCD display with I2C backpack (optional, for displaying readings without a computer) (Pixel Electronics, pixelelectronics.co.ke)
- 4 x Male-to-female jumper wires (only needed if using the LCD display) (Nerokas, nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (school-supplied)

HOW IT WORKS

Imagine the DHT11 sensor as a tiny weather station that can feel how hot or cold the air is and how much water vapour (humidity) is floating around. Inside the sensor there is a special material that changes its electrical behaviour depending on the temperature and humidity around it. The sensor measures these tiny changes and turns them into numbers.

The DHT11 talks to the Arduino using a single data wire and a special timing pattern - it sends a burst of electrical pulses that represent the temperature and humidity as a code. This is different from a simple sensor that just gives a smooth voltage; the DHT11 sends a structured digital message, a bit like tapping out Morse code.

To understand this code, we use a "library" - a pre-written piece of software that knows exactly how to listen to those pulses and translate them into a temperature reading we can understand, like 24 degrees Celsius. Our Arduino sketch simply asks the library "what's the temperature?" every couple of seconds, and the library does the hard work of decoding the sensor's signal.

Once we have the number, we can display it. The simplest way is to print it to the Serial Monitor, a text window on the computer that talks to the Arduino over the USB cable. For an added challenge, we can also show the number on a small screen (an LCD) so the project works without being plugged into a laptop.

STEP-BY-STEP INSTRUCTIONS

1. Place the breadboard in front of you and insert the DHT11 sensor module (breakout board version) into it, ensuring its three pins are in three separate rows.
2. Identify the three pins on the DHT11 breakout: VCC (power), GND (ground), and OUT/DATA/SIG (signal). Most breakout boards label these clearly.
3. Connect a jumper wire from the DHT11's VCC pin to the Arduino's 5V pin.
4. Connect a jumper wire from the DHT11's GND pin to any GND pin on the Arduino.
5. Connect a jumper wire from the DHT11's OUT/DATA/SIG pin to digital pin 2 on the Arduino.
6. Double-check all connections match the wiring diagram before connecting the USB cable - loose wires are the most common cause of "no reading" errors.
7. Connect the Arduino to the computer with the USB cable.
8. Open the Arduino IDE. Go to Tools > Manage Libraries, search for "DHT sensor library" by Adafruit, and install it along with its dependency "Adafruit Unified Sensor".
9. Open a new sketch and type (or paste) the Full Code provided below.
10. Select the correct board (Arduino Uno) and port under the Tools menu.
11. Click Upload. Wait for the "Done uploading" message at the bottom of the IDE window.
12. Open the Serial Monitor (magnifying glass icon, top right) and set the baud rate to 9600.
13. Observe the temperature readings appearing every 2 seconds. Ask students to breathe gently on the sensor or hold it in their palm and watch the reading rise.
14. If an LCD is being used, wire its SDA pin to Arduino's A4, SCL pin to A5, VCC to 5V, and GND to GND, then re-upload the extended code version that includes LCD output (see Extension Challenges).
15. Discuss with students why the readings might fluctuate slightly even in a still room (sensor precision limits, air currents, body heat nearby).

FULL CODE

```
// Temperature Logger using DHT11 sensor
// Reads temperature and humidity, prints to Serial Monitor
```

```
#include <DHT.h>          // Include the DHT sensor library

#define DHTPIN 2          // The DHT11 data pin is connected to digital pin 2
#define DHTTYPE DHT11     // Tell the library we are using the DHT11 model (not DHT22)

DHT dht(DHTPIN, DHTTYPE);  // Create a DHT object called "dht" using our pin and type

void setup() {
  Serial.begin(9600);     // Start serial communication at 9600 bits per second
  dht.begin();             // Initialise the sensor so it's ready to take readings
  Serial.println("Temperature Logger Starting..."); // Friendly startup message
}

void loop() {
  delay(2000);             // DHT11 can only be read reliably every 2 seconds - do not remove this delay

  float humidity = dht.readHumidity();       // Ask the sensor for humidity (%RH)
  float temperature = dht.readTemperature(); // Ask the sensor for temperature in Celsius

  // Check if either reading failed (sensor returns "nan" = not a number on failure)
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor! Check wiring."); // Warn the user
    return;                // Skip the rest of this loop and try again next time
  }

  Serial.print("Temperature: ");
  Serial.print(temperature);  // Print the temperature value
  Serial.print(" C  |  Humidity: ");
  Serial.print(humidity);     // Print the humidity value
  Serial.println(" %");       // End the line and move to a new one
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
Serial Monitor shows "Failed to read from DHT sensor!" repeatedly	Loose or incorrect wiring, especially the data pin	Recheck all three connections; ensure data pin matches DHTPIN in code (pin 2)
Serial Monitor shows garbled or no text at all	Wrong baud rate selected in Serial Monitor	Set Serial Monitor baud rate to match Serial.begin(9600) in the code, i.e. 9600
Code fails to compile with "DHT.h: No such file or directory"	DHT sensor library not installed	Install "DHT sensor library" and "Adafruit Unified Sensor" via Library Manager
Temperature reading seems stuck at the same value	DHT11 is being read faster than its 2-second minimum refresh rate	Ensure the delay(2000) is present and not shortened
Reading is unusually high compared to room temperature	Sensor is placed too close to the Arduino board or USB port, which generate heat	Move the sensor a few centimetres away from the board and other heat sources

DISCUSSION QUESTIONS

- What two things does the DHT11 sensor actually measure?
- Why do we need a "library" in our code instead of writing everything ourselves?
- Why does the code have a 2-second delay before each reading - what might happen if we removed it?
- Can you think of a real-world situation where knowing the temperature automatically (without a person checking) would be useful?
- If you wanted to know the temperature outside your house at 3am without waking up, how could this project help you?

EXTENSION CHALLENGES

- **Easy:** Modify the code to also print the humidity as a percentage on its own separate line, clearly labelled.
- **Medium:** Add an LED that lights up automatically whenever the temperature rises above 28°C, using an if statement and a digital output pin.
- **Hard:** Connect a 16x2 I2C LCD display and modify the code so both temperature and humidity are shown on the screen in real time, updating every 2 seconds, without needing the Serial Monitor at all.

7 Fan control

ADVANCED · ~90 MIN · BANDS: 13-15 YRS (12) · 2 GROUP(S)

MATERIALS

- 1 x Arduino Uno R3 (Nerokas, nerokas.co.ke)
- 1 x TMP36 analogue temperature sensor (Nerokas, nerokas.co.ke)
- 1 x 5V single-channel relay module (Pixel Electronics, pixelelectronics.co.ke)
- 1 x 12V DC cooling fan (small computer/car-style fan) (Nerokas, nerokas.co.ke)
- 1 x 12V DC power adapter or 12V battery pack (separate from Arduino power) (Nerokas, nerokas.co.ke)
- 1 x Half-size or full-size breadboard (Pixel Electronics, pixelelectronics.co.ke)
- 1 x Pack of male-to-male jumper wires (Pixel Electronics, pixelelectronics.co.ke)
- 1 x LED (any colour, for fan-status indicator) (Nerokas, nerokas.co.ke)
- 1 x 220Ω resistor (for the LED) (Nerokas, nerokas.co.ke)
- 1 x USB cable (Arduino to computer) (Nerokas, nerokas.co.ke)

- 1 x Laptop or computer with Arduino IDE installed

HOW IT WORKS

A temperature sensor called the TMP36 works a bit like a tiny thermometer that speaks in electricity instead of numbers. As it gets warmer, the voltage coming out of it goes up a tiny, predictable amount. The Arduino reads this voltage and does some maths to turn it into a temperature in degrees Celsius.

The Arduino cannot power a fan directly from one of its pins — the fan needs more current than the Arduino can safely provide, and often a different voltage too (12V instead of 5V). This is where the **relay** comes in. A relay is an electrically controlled switch. When the Arduino sends a small signal to the relay, a tiny electromagnet inside it clicks a switch closed, which lets a completely separate, more powerful circuit (the 12V fan and its own power supply) turn on. The Arduino and the fan circuit never touch electrically — they are just linked by that click.

Put together: the Arduino constantly checks the temperature. If it gets too warm, the Arduino tells the relay to "click on," which powers the fan. When it cools down again, the Arduino tells the relay to "click off." We also build in a small gap between the "turn on" and "turn off" temperatures (called hysteresis) so the fan does not flicker on and off rapidly right at the threshold.

STEP-BY-STEP INSTRUCTIONS

1. Place the Arduino, breadboard, TMP36, relay module, and LED on the working area. Do not connect the 12V power supply yet.
2. Insert the TMP36 into the breadboard. Note it has 3 legs: usually left = VCC (5V), middle = signal (OUT), right = GND — check the flat face orientation printed on your specific sensor before wiring.
3. Connect TMP36 VCC to Arduino 5V, TMP36 GND to Arduino GND, and TMP36 OUT to Arduino analogue pin A0.
4. Connect the LED's long leg through the 220Ω resistor to Arduino digital pin 6, and the short leg to GND. This LED will show when the fan is "on."
5. Connect the relay module: its VCC pin to Arduino 5V, its GND pin to Arduino GND, and its IN (signal) pin to Arduino digital pin 7.
6. On the relay module's other side (the high-power terminals, usually marked COM, NO, NC), connect the fan and the 12V supply in series through the COM and NO terminals: 12V supply positive → relay COM, relay NO → fan positive, fan

negative → 12V supply negative. Do NOT connect this side to the Arduino's power at all — it must be a separate circuit.

7. Double-check all wiring before connecting the 12V power supply. Ask a teacher/facilitator to verify wiring at this stage.
8. Connect the Arduino to the computer via USB and upload the code below.
9. Open the Serial Monitor (9600 baud) and observe the live temperature readings printed every second.
10. Now plug in the 12V power supply for the fan circuit.
11. Warm the TMP36 gently by cupping it in your hand or holding a warm object near it (not directly on the sensor). Watch the Serial Monitor temperature rise. When it crosses the "on" threshold, students should hear the relay click and see the fan start spinning and the LED light up.
12. Let the sensor cool down (blow on it gently or remove your hand). Watch the temperature fall. When it crosses the "off" threshold, the relay should click again, the fan should stop, and the LED should turn off.
13. Discuss what was observed, then try adjusting the threshold values in the code and re-uploading to see the effect.

FULL CODE

```
const int tempPin = A0;        // TMP36 analogue output is connected to A0
const int relayPin = 7;        // Digital pin that signals the relay module
const int ledPin = 6;          // Status LED, lights up when fan is on

float onThreshold = 28.0;      // Temperature (°C) at which the fan switches ON
float offThreshold = 26.0;     // Temperature (°C) at which the fan switches OFF
                               // The gap between these two values is called hysteresis -
                               // it stops the fan flickering on/off rapidly near one value

bool fanIsOn = false;          // Keeps track of whether the fan is currently on

void setup() {
  pinMode(relayPin, OUTPUT);    // Relay pin sends signals, so it is an output
  pinMode(ledPin, OUTPUT);      // Same for the LED
  digitalWrite(relayPin, LOW);  // Make sure fan starts OFF when the sketch begins
  digitalWrite(ledPin, LOW);
  Serial.begin(9600);           // Start serial communication so we can watch readings
}

void loop() {
  int rawValue = analogRead(tempPin);            // Read raw sensor value, 0-1023
  float voltage = rawValue * (5.0 / 1023.0);     // Convert that number into a real voltage (0-5V)
  float temperatureC = (voltage - 0.5) * 100.0;  // TMP36 formula: 0°C = 0.5V, then 10mV per °C

  Serial.print("Temperature: ");
  Serial.print(temperatureC);
```

```
  Serial.println(" C");                          // Print reading so we can watch it live

  if (!fanIsOn && temperatureC >= onThreshold) {
    // Only turns on if it was previously off AND we've crossed the "on" line
    fanIsOn = true;
    digitalWrite(relayPin, HIGH);    // Energise the relay coil, which closes its switch
    digitalWrite(ledPin, HIGH);      // Turn on the status LED
    Serial.println("Fan turned ON");
  }
  else if (fanIsOn && temperatureC <= offThreshold) {
    // Only turns off if it was previously on AND we've dropped below the "off" line
    fanIsOn = false;
    digitalWrite(relayPin, LOW);     // De-energise the relay coil, opening the switch
    digitalWrite(ledPin, LOW);       // Turn off the status LED
    Serial.println("Fan turned OFF");
  }

  delay(1000);   // Wait one second before checking again - keeps the Serial Monitor readable
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
Fan never turns on, even when it feels warm	Threshold value set too high for the room, or TMP36 wired incorrectly so it reads odd values	Check Serial Monitor readings look sensible (room temperature ~20-25°C); lower the onThreshold to test; verify TMP36 pin orientation
Relay clicks (you hear it) but the fan does not spin	Fan's power circuit (12V side) is not wired correctly, or the 12V supply is not connected/ switched on	Check COM/NO wiring on relay, confirm 12V supply is plugged in and polarity is correct
Fan flickers rapidly on and off near the threshold	onThreshold and offThreshold are set to the same value, removing the hysteresis gap	Make sure offThreshold is a couple of degrees lower than onThreshold
Temperature readings on Serial Monitor look wrong (very negative, or stuck at one value)	TMP36 legs wired to the wrong pins, or loose breadboard connection	Re-check VCC/OUT/GND against the sensor's datasheet pinout; reseat all jumper wires
Arduino resets or behaves oddly when the relay switches	Relay module drawing too much current from the Arduino's 5V pin, or missing flyback protection	Confirm the relay module has its own onboard protection diode; consider powering the relay module from a separate 5V source if problems persist

DISCUSSION QUESTIONS

- What job does the TMP36 sensor do, and how does it "tell" the Arduino the temperature?

- Why can't the Arduino power the fan directly from one of its own pins?
- What would happen if we set the onThreshold and offThreshold to exactly the same number? Why is that a problem?
- Can you think of a real building or machine that might use a temperature-controlled fan like this one?
- If you wanted the fan to spin faster as it got hotter (instead of just on/off), what part of the circuit would need to change?

EXTENSION CHALLENGES

- **Easy:** Replace the fixed threshold values in the code with a potentiometer connected to another analogue pin, so students can physically turn a knob to set the temperature at which the fan turns on.
- **Medium:** Add a 16x2 LCD screen (or reuse the Serial Monitor creatively) to continuously display the current temperature and fan status ("FAN ON" / "FAN OFF") in a friendly readable format.
- **Hard:** Replace the simple on/off relay control with a MOSFET and PWM signal so the fan speed increases gradually as temperature rises, rather than switching fully on or off — turning this into a proportional cooling system.

8 Simon says game

ADVANCED · ~120 MIN · BANDS: 13-15 YRS (12) · 2 GROUP(S)

MATERIALS

- 1 x Arduino Uno R3 (or compatible) (Nerokas)
- 1 x Solderless breadboard, 830 tie-points (Nerokas)
- 4 x LEDs, 5mm — 1 red, 1 green, 1 yellow, 1 blue (Pixel Electronics)
- 4 x Push buttons, 12mm tactile (Pixel Electronics)
- 1 x Piezo buzzer, passive (Nerokas)
- 4 x Resistors, 220 ohm, for LEDs (Nerokas)
- 4 x Resistors, 10k ohm, for button pull-downs (Nerokas)
- Jumper wires, male-to-male, approx. 25 pieces (Nerokas)
- 1 x USB A-to-B cable, for programming and power (Nerokas)
- 1 x 9V battery with barrel connector (optional, for portable play) (Pixel Electronics)

HOW IT WORKS

This project is a memory game. The Arduino "computer brain" picks a random colour, lights up the matching LED, and plays a tone on the buzzer. The player must press the button that matches. If they get it right, the Arduino adds one more colour to the pattern and repeats the whole sequence — now one step longer. The game keeps going, one colour longer each round, until the player presses the wrong button.

Three big ideas make this work:

- **Arrays** — a list that stores the growing sequence of colours, like a shopping list that gets one more item added each round.
- **Random numbers** — the Arduino uses a random number generator to decide which colour comes next, so no two games are the same.
- **Input and output together** — the Arduino both sends signals out (to light LEDs and buzz the buzzer) and reads signals in (from the buttons) in the same program, comparing what the player pressed with what it expected.

Pull-down resistors are used on the buttons so that when a button is not pressed, the Arduino reliably reads a clean LOW signal instead of a random "floating" value.

STEP-BY-STEP INSTRUCTIONS

1. Place the Arduino next to the breadboard so power rails are easy to reach.
2. Connect Arduino 5V to the breadboard's red (+) power rail, and Arduino GND to the blue (-) power rail.
3. Insert the 4 LEDs into the breadboard, spaced apart, each with the long leg (anode) facing the same direction.
4. Connect a 220 ohm resistor from each LED's long leg to a digital pin: Red LED to pin 2, Green LED to pin 3, Yellow LED to pin 4, Blue LED to pin 5.
5. Connect each LED's short leg (cathode) to the GND rail.
6. Insert the 4 push buttons into the breadboard, one per colour, away from the LEDs.
7. For each button: connect one leg to 5V, and the opposite leg to both a digital pin (pin 8, 9, 10, 11 respectively) and, via a 10k resistor, to GND. This forms the pull-down.
8. Connect the buzzer's positive leg to pin 12 and its negative leg to GND.

9. Double-check every LED and button pin matches the numbers used in the code (students should trace each wire with a finger and say the pin number out loud).
10. Connect the USB cable and upload the sketch below.
11. Watch the "power-on" light show: all four LEDs should flash once together — this confirms wiring is correct before the game starts.
12. Observe the Arduino play a 1-colour sequence. Ask students to predict what happens if they press the wrong button.
13. Let students take turns playing. Encourage them to say each colour out loud before pressing, to link listening, seeing, and pressing.
14. When a student loses, discuss what score (sequence length) they reached and let the next student try to beat it.
15. Once the game works, ask students to identify which part of the code is the array, which part is input, and which part is output.

FULL CODE

```
// Simon Says Game (Advanced)
// Classic memory game with LEDs, buttons and a buzzer

const int NUM_COLOURS = 4;                      // how many colours/buttons we have
const int ledPins[NUM_COLOURS]    = {2, 3, 4, 5};    // LED pin for each colour
const int buttonPins[NUM_COLOURS] = {8, 9, 10, 11};  // button pin for each colour
const int buzzerPin = 12;                        // buzzer output pin

// each colour gets its own buzzer tone frequency (in Hz)
const int tones[NUM_COLOURS] = {262, 330, 392, 494}; // C, E, G, B notes

int gameSequence[100];   // stores the full sequence for this game (max 100 rounds)
int currentLevel = 0;    // how many steps long the sequence currently is

void setup() {
  Serial.begin(9600);                // for debugging / showing score in Serial Monitor

  for (int i = 0; i < NUM_COLOURS; i++) {
    pinMode(ledPins[i], OUTPUT);       // LEDs are outputs
    pinMode(buttonPins[i], INPUT);     // buttons are inputs (external pull-down resistor used)
  }
  pinMode(buzzerPin, OUTPUT);

  randomSeed(analogRead(A0));          // seed randomness using electrical noise on unused pin A0

  powerOnLightShow();                  // quick flash of all LEDs to confirm wiring
}

void loop() {
  addNewStepToSequence();              // grow the sequence by one random colour
  playSequence();                      // show the player the sequence so far
  bool playerCorrect = getPlayerSequence(); // let player try to repeat it
```

```
  if (!playerCorrect) {
    gameOverEffect();                 // play a sad sound/light if they got it wrong
    Serial.print("Game over! You reached level: ");
    Serial.println(currentLevel - 1);  // report score reached before the loss
    currentLevel = 0;                 // reset for a new game
    delay(2000);                      // pause before starting again
  }
}

// Adds one new random colour to the end of the sequence array
void addNewStepToSequence() {
  gameSequence[currentLevel] = random(0, NUM_COLOURS); // pick a random index 0-3
  currentLevel++;                                      // sequence is now one longer
}

// Lights up and buzzes each colour in the current sequence, in order
void playSequence() {
  delay(500); // short pause before playback starts
  for (int i = 0; i < currentLevel; i++) {
    int colour = gameSequence[i];       // look up which colour this step is
    lightAndBuzz(colour, 400);          // show it to the player
    delay(200);                         // small gap between flashes
  }
}

// Turns on one LED and plays its tone for a given duration (ms), then turns it off
void lightAndBuzz(int colour, int duration) {
  digitalWrite(ledPins[colour], HIGH);
  tone(buzzerPin, tones[colour]);       // start playing this colour's note
  delay(duration);
  digitalWrite(ledPins[colour], LOW);
  noTone(buzzerPin);                    // stop the tone
}

// Waits for the player to press buttons matching the sequence
// Returns true if they get the whole sequence right, false if they make a mistake
bool getPlayerSequence() {
  for (int i = 0; i < currentLevel; i++) {
    int pressedColour = waitForButtonPress(); // wait until a button is pressed
    lightAndBuzz(pressedColour, 300);         // feedback: light up what they pressed

    if (pressedColour != gameSequence[i]) {
      return false;                           // wrong button -> player loses
    }
  }
  return true; // matched every step correctly
}

// Waits (blocks) until exactly one button is pressed, then returns its colour index
int waitForButtonPress() {
  while (true) {
    for (int i = 0; i < NUM_COLOURS; i++) {
      if (digitalRead(buttonPins[i]) == HIGH) { // button pressed pulls pin HIGH
        delay(20);                              // simple debounce delay
        while (digitalRead(buttonPins[i]) == HIGH) {
          // wait here until player releases the button
        }
        return i;                               // return which colour was pressed
      }
    }
```

```
  }
}

// Flashes all LEDs together once, to show the game has powered on correctly
void powerOnLightShow() {
  for (int i = 0; i < NUM_COLOURS; i++) {
    digitalWrite(ledPins[i], HIGH);
  }
  delay(300);
  for (int i = 0; i < NUM_COLOURS; i++) {
    digitalWrite(ledPins[i], LOW);
  }
  delay(300);
}

// Plays a descending "sad trombone" style tone sequence when the player loses
void gameOverEffect() {
  for (int i = 0; i < NUM_COLOURS; i++) {
    digitalWrite(ledPins[i], HIGH); // flash all LEDs together
  }
  tone(buzzerPin, 200);   // low sad tone
  delay(300);
  tone(buzzerPin, 150);
  delay(300);
  noTone(buzzerPin);
  for (int i = 0; i < NUM_COLOURS; i++) {
    digitalWrite(ledPins[i], LOW);
  }
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
Buttons register presses randomly, even when not touched	Missing or wrongly placed 10k pull-down resistor, causing pin to "float"	Check each button leg goes to GND through a 10k resistor, not left disconnected
Wrong LED lights up for a given colour	ledPins array order does not match physical wiring	Trace each wire from Arduino pin to LED and confirm it matches the array order in code
Game seems to always play the same sequence each time it is powered on	randomSeed() is missing, or A0 pin is connected to something (removing electrical noise)	Ensure randomSeed(analogRead(A0)) is called once in setup, and leave A0 unconnected
Button presses are counted twice, or level skips ahead unexpectedly	No debounce — a single physical press is read as multiple electrical presses	Confirm the debounce delay(20) and "wait for release" loop are present in waitForButtonPress()
Buzzer stays silent	Buzzer wired backwards, or on the wrong pin, or a passive buzzer used incorrectly with tone()	Check buzzer polarity and pin 12 connection; confirm buzzer is passive type (works with tone command)

Sequence plays but player's correct presses are still marked wrong	Button pin numbers in buttonPins array do not match the physical wiring order used for colours	Match button-to-colour order exactly the same way LEDs are matched to colours

DISCUSSION QUESTIONS

- What part of the code stores the growing list of colours, and why is an array a good tool for that job?
- Why does the game use a pull-down resistor on each button instead of connecting the button directly to the pin?
- How does the Arduino decide which colour comes next — is it truly random, or just "hard to predict"?
- What would happen to the game if we removed the debounce delay in waitForButtonPress()? Why?
- Can you think of a real-world device that also needs to remember and check a sequence of inputs, like a lock or a password screen?

EXTENSION CHALLENGES

- **Easy:** Change the LED colours' buzzer tones so each one plays a different, more musical note (look up frequency values for notes online).
- **Medium:** Add a scoring display using the Serial Monitor that shows the player's current level as they play, not just at game over.
- **Hard:** Add a "speed mode" where the delay between flashes gets shorter every 5 levels, making the game progressively harder to follow.

9 Line-following robot

ADVANCED · ~120 MIN · BANDS: 13-15 YRS (12) · 2 GROUP(S)

MATERIALS

- Arduino Uno R3 — 1 unit (Nerokas, nerokas.co.ke)
- L298N Dual H-Bridge Motor Driver Module — 1 unit (Nerokas, nerokas.co.ke)
- IR Infrared Line-Tracking Sensor Modules (digital output, with onboard sensitivity potentiometer) — 2 units (Pixel Electronics, pixelelectronics.co.ke)
- TT Gear DC Motors, yellow, 3–6V — 2 units (Nerokas, nerokas.co.ke)
- Two-wheel robot chassis kit (includes wheels and rear caster/support ball) — 1 unit (Nerokas, nerokas.co.ke)

- 4×AA battery holder with on/off switch — 1 unit (Pixel Electronics, pixelelectronics.co.ke)
- AA batteries — 4 units (Pixel Electronics, pixelelectronics.co.ke)
- Jumper wire pack (male-to-male and male-to-female) — 1 pack (Nerokas, nerokas.co.ke)
- Small breadboard (optional, for testing sensors before final wiring) — 1 unit (Pixel Electronics, pixelelectronics.co.ke)
- USB-A to USB-B cable (for programming the Arduino) — 1 unit (Nerokas, nerokas.co.ke)
- Black insulation tape (to build the practice track line) — 1 roll (Nerokas, nerokas.co.ke)
- White poster board, vinyl sheet, or smooth floor tile (track background)
- Small Phillips screwdriver and M3 screws (usually included with chassis kit) — 1 set (Nerokas, nerokas.co.ke)
- Zip ties or Velcro strips for tidying wiring — a few (Pixel Electronics, pixelelectronics.co.ke)

HOW IT WORKS

This robot uses two "eyes" that can only see one thing: how much light bounces back off the floor. These are called **infrared (IR) sensors**. Each sensor shines an invisible beam of light down at the floor and measures how much of it reflects back. White surfaces bounce a lot of light back, so the sensor reports "no line here." Black surfaces soak up (absorb) most of the light instead of bouncing it, so the sensor reports "line detected!"

The Arduino checks both sensors many times every second. Based on which sensor (left, right, both, or neither) currently sees the black line, it decides what the robot should do next: drive straight, steer left, steer right, or stop because it has lost the track completely.

The Arduino itself is too weak to power motors directly — its pins can only supply a tiny trickle of electricity, nowhere near enough to spin a motor with any force. That's why we use a **motor driver board** (the L298N). Think of it as a set of electronic light switches: the Arduino sends a small "instruction" signal to the driver, and the driver uses power from the battery pack to actually turn the motors on, off, or run them at different speeds. This separation — small brain signals

controlling big battery power — is the same idea used in real cars, robots, and even elevators.

By constantly comparing what the two sensors see and nudging the motors left or right, the robot performs hundreds of tiny corrections per second, which is what makes it appear to "follow" the line smoothly.

STEP-BY-STEP INSTRUCTIONS

1. Assemble the chassis kit: attach the two TT motors and wheels to the frame, and fix the caster ball at the front or back for balance. Do not wire anything yet — get the mechanical build solid first.
2. Mount the L298N motor driver board flat on top of the chassis using screws or double-sided tape, keeping its terminals easily accessible.
3. Mount the two IR sensor modules at the very front of the chassis, close to the ground (a few millimetres above the surface), spaced a few centimetres apart so the black line can pass between or under them as the robot moves.
4. Wire the left motor to the OUT1/OUT2 terminals of the L298N and the right motor to OUT3/OUT4. Note which motor is "left" and which is "right" as seen from behind the robot — this matters for the code later.
5. Connect the L298N control pins to the Arduino: ENA to pin 5, IN1 to pin 8, IN2 to pin 9, IN3 to pin 10, IN4 to pin 11, and ENB to pin 6. Connect the L298N GND to the Arduino GND — this shared ground is essential for the signals to make sense.
6. Connect the battery pack's positive and negative wires to the L298N's power input terminals (12V/VCC and GND), and turn the battery switch off for now.
7. Wire each IR sensor: VCC to 5V, GND to GND, and the digital OUT pin to Arduino pin 2 (left sensor) and pin 3 (right sensor).
8. Before uploading the full program, upload a simple test sketch that just prints sensor readings to the Serial Monitor. Slide a piece of black tape under each sensor and observe whether the reading is HIGH or LOW — this tells you whether your specific sensor reports HIGH or LOW for "black detected." Note this down, since it decides whether the main code logic needs adjusting.
9. Adjust the small potentiometer screw on each IR sensor module while watching the onboard LED indicator: turn it until the LED clearly switches state right at the boundary between the black tape and the white background. This calibration

step is critical — students should observe how sensitivity changes what counts as "line" versus "background."

10. Upload the full line-following sketch (see Full Code section) to the Arduino, matching HIGH/LOW to what was observed in the test step.
11. Build a simple oval or figure-eight practice track using black tape on a white board or floor sheet. Include at least one gentle curve and one sharper turn.
12. Place the robot on the track with the line running between its two sensors, turn on the battery switch, and observe. It should creep forward and wiggle gently left and right as it follows the curve of the line.
13. If the robot swerves wildly or drives off the line immediately, pause and check sensor calibration first, then check motor direction wiring (see Common Mistakes table).
14. Once it works reliably, invite students to test on sharper curves, gaps in the line, and intersections to see where the robot struggles.

FULL CODE

```
// ===== Line-Following Robot (Advanced) =====
// Two IR sensors detect a black line on a white track.
// The Arduino steers two motors via an L298N motor driver
// to keep the line centered between the sensors.

// --- Pin
```

Materials — Scaling

Each topic's materials list above is **per group**. Multiply it by the number of groups that run the topic (shown on each card) to get the totals to prepare. Shared kit (laptops, power strips, extension leads) can rotate between groups rather than be bought per group.

Safety, Logistics & Contingencies

- **Space:** one table per group with room to move; keep walkways clear and power leads taped down.

- **Movement:** groups start and stop together on the master timetable — stagger the break and lunch queues so the space never crowds.
- **First aid & allergies:** collect allergy info at registration; the first-aider holds the kit and the contact list all day.
- **Wet weather:** move outdoor free-play to the hall; keep one indoor back-up game ready.
- **Early finishers:** use each project's extension challenges (listed in the project pages below).
- **Equipment failure:** keep one spare kit per band; a failed build becomes a debugging lesson, not a lost hour.

Staff Briefing Checklist

- [ ] Every facilitator knows their group, their band, and their room.
- [ ] Kits counted and laid out per group before doors open.
- [ ] Registration desk has the sign-in sheet, badges, and allergy list.
- [ ] Timekeeper owns the master timetable and calls transitions out loud.
- [ ] Floating helpers know which groups they cover during breaks.
- [ ] Showcase space and closing certificates ready before the last block.